PREPARED BY

CARTRACK INSURANCE

CLASSIFICATION

INTERNAL

HANDOFF DOCUMENT 1 OF 4 · 21 SEPTEMBER 2026

The RM System

How the app is built, how the data is shaped, and why six pieces of it that look strange must not be removed. Every one of them exists because something was lost once.

SINGLE FILE	TABLES	LOAD BEARING
index.html	**7 exist, 5 live**	**6 decisions**
3,300 lines, no build step	users and activity are unused	each one from a real incident

IN THIS DOCUMENT

1 What it is
2 Roles and surfaces
3 The data model
4 The write path
5 The six decisions that look odd and are load-bearing
6 Recovery reads two records, and neither is enough alone
7 Do not do these things
8 Known state and open items
9 Changes since the first issue (21 Sept 2026)

THE HANDOFF PACK

● 1 · RM System | 2 · Security position | 3 · Comparison method | 4 · Portal build spec

SECTION

Continued

Cartrack Insurance, Commercial Division. Handoff document 1 of 4. Updated 22 October 2026. Source read at `cartrack-rm-system@ba7b48d`, app version `v2026.09.22a`. First issued 21 September 2026 against `2959ea2` / `v2026.08.27a`; what changed since is in section 9.

Read section 5 before you change anything. Most of this file's strangest code is load-bearing, and each piece of it is there because something was lost once.

SECTION

1. What it is

A **single HTML file**. `cartrack-rm-system/index.html`, about 3,300 lines and 415 KB, containing the markup, the CSS, the entire application, the seed data and two base64 logos. There is no build step, no bundler, no framework, no npm, no back end of ours.

- **Hosting:** GitHub Pages off `main`. **Deploying is committing and pushing to `main`.** Pages rebuilds in one to two minutes.
- **Data path:** browser → `supabase-js` (CDN, `defer`) → Supabase PostgREST → Postgres. Nothing of ours sits in between. See document 2 before you decide whether that is acceptable.
- **Offline shell:** `sw.js`, a network-first service worker, cache `ct-rm-shell-v11`. It caches only same-origin GETs for the app shell. **Supabase calls and fonts pass straight through and are never cached.** Data is never in the service worker cache.
- **Installable:** `manifest.webmanifest`, home-screen icons in `assets/`. Most RMs run it as a phone app, which is why so much of the code is about surviving a backgrounded tab.
- **Version stamp:** `const APP_VERSION` is printed on the sign-in card. **Bump it on every push.** It is the only way support can tell what a given phone is running.

There is no Netlify anywhere in this stack. There is no `netlify.toml`, no build, no functions.

Rendering model

One global mutable `S` object and a `render()` function that rewrites `root.innerHTML` for the whole screen, then re-attaches event handlers in a matching `wireX()` function. Every view is a pair: `cmdClients()` builds the HTML, `wireCmdClients()` binds it. If you add a view, add both, or the buttons silently do nothing.

There is no virtual DOM and no diffing. That is why background refresh has to be so careful about when it is allowed to run (section 4.6).

SECTION

2. Roles and surfaces

Four roles in `S.role`, chosen at the gate:

ROLE	WHO	SURFACES
`rm`	the seven RMs	My Day, My Pipeline, My Clients, My Commission, Leaderboard, **Missing work** (read only)
`mgr`	RM manager	Team progress, book of business, problem areas
`cmd`	Command View, management	All RMs, overview, clients, pipeline, house account, problems, changes, **Recovery**, settings
`pay`	Payroll	Enter commission, payout summary, copy for accountant

Recovery is Command View only, deliberately. An RM never sees a restore button. They get a read-only "Missing work" tab that shows their own missing leads and Active Clients with a copy-to-WhatsApp button. The judgement about what should come back is the RM's; the write is management's. Telling an RM to "open Recovery" is wrong, they do not have it.

Sign-in is one of three paths: an RM picks their name and enters a six-digit PIN; a manager enters a hard-coded default passcode; or an operator enters a master key, which opens **any** surface without touching anyone's PIN. Master keys live in a map in the source so they can be rotated individually. All of this is client side. Document 2 deals with what that means.

SECTION

3. The data model

3.1 The key to table router

3. The data model

The app was written against a flat key-value store and still talks that way. Everything goes through `sGet(key)` and `sSet(key, value)` where a key is a string and the value is JSON. `_route(key)` maps each key pattern onto a real Postgres table:

KEY PATTERN	TABLE	SHAPE
`deals:<slug>`	`deals`	array of deal objects, one row per deal
`deals:__house`	`deals`	the house account, `rm_slug = '__house'`
`portfolio:<slug>`	`portfolio`	array of Active Client objects, one row per client
`payroll:<YYYY-MM>`	`payroll`	one row per month, `rows` and `salary` as JSON
`payrolls:`	`payroll`	**every** month, oldest first, for Recovery
`config:pins`	`config`	key `pins`, the RMs' PIN hashes
`config:logins`	`config`	key `logins`, last-seen per user
`config:meta`	`config`	key `meta`
`config:snapshot:<key>`	`config`	pre-overwrite snapshots, see 5.4
`changes:<date>:<slug>`	`config`	the change journal, **one row per RM per day**
`lost:<slug>`	`config`	Not Successful leads, one JSON array per RM

So **seven tables exist and five are live**: `orgs`, `deals`, `portfolio`, `payroll` and `config`. `users` and `activity` exist in the database but nothing in the app reads or writes them. Do not assume they are maintained.

Everything is scoped by `org_id`, resolved once by `resolveOrg()` which looks up `orgs` by `fsp_no = '17266'` and caches the id. One organisation in practice.

3.2 Why so much lives in `config`

`config` is a generic `(org_id, key, value jsonb)` table with a unique constraint on `(org_id, key)`. Lost leads, the change journal, PINs and snapshots all live there rather than in their own tables. That was expedient and it has a real cost: **lost leads and journal entries do not get the merge-safe per-row save** that `deals` and `portfolio` get, they are whole-blob upserts with a hand-written merge (`_sbSet`, `kind === 'lost'`). If you promote anything out of `config` into its own table, keep the merge behaviour.

The journal key is `changes:<date>:<slug>`, **one row per RM per day**, and that matters. It used to be a single shared `changes:<date>` row, and two RMs saving at roughly the same moment overwrote each other's entries, so moves went missing from the morning report. Reads aggregate with `like('key', 'changes:<date>%')`, which also still picks up the old shared rows.

3.3 Column mapping

`_dealRowToApp` / `_dealAppToRow` and `_portRowToApp` / `_portAppToRow` translate between snake_case columns and the camelCase the app uses. Two things in there are not decoration:

- **Ids are real UUIDs**, generated client-side with `crypto.randomUUID()`. A new item's id in the app **is** its database row id. The merge-safe save depends on that to upsert per row instead of rewriting a list. A non-UUID id is treated as legacy and inserted so the database assigns one.
- **`broker_fee` is tolerated as absent.** `_hasBrokerFeeCol` is learned from the first server row seen; if a write is rejected with a `broker_fee` schema error the field is stripped and the write is retried once. The column was added on 30 July 2026 and the tolerance stayed. Golden rule: a missing column must never break saving everything else.

SECTION

4. The write path

This is the part to understand before touching anything.

4. The write path

```
sSet(key, value)
  -> _guardWrite       refuse destructive writes outright        (5.3)
  -> _snapshot         copy the previous value aside             (5.4)
  -> _memSet           mirror locally, always
  -> _obPut            durable outbox flag BEFORE the network    (4.3)
  -> _writeChain       serialise writes per key                  (4.4)
       -> _sbSetAdopt -> _sbSet -> _mergeSaveTable               (4.2)
       -> _obClear on confirmed success
       -> _pendingWrites + red banner on failure                 (4.5)
```

4.1 `sGet` and `_readOk`

`sGet` reads from Supabase and falls back to an in-memory `_mem` copy on failure. It also records `_readOk[key]` : **was this key's value produced by a read the server actually answered?** That flag exists for exactly one caller, `seedIfEmpty` , and it is the fix for the August data loss (5.2).

4.2 Merge-safe saves, and `_known`

`_mergeSaveTable` replaced a delete-everything-then-reinsert save. The old one meant a tab holding stale data silently reverted every change made on any other device since that tab last loaded. Now:

- `_known[key]` is the set of row ids **this session actually saw on the server**.
- On save, it deletes only ids that are in `_known` , absent from the outgoing list, and present on the server. **A stale tab therefore cannot wipe rows another device added after our last fetch.**
- A row the app is sending that the server does not have but `_known` does is skipped: it was deleted on another device and stays deleted.
- For `deals` , a server row with a newer `updated_at` than ours wins and is not overwritten.
- After the write it re-reads and returns the merged truth, which `_adoptMerged` pushes back into `S` so the UI shows what everyone sees.

If you add a table, give it this treatment. Do not add a delete-all-then-insert path.

4.3 The durable outbox

`localStorage` key `ct_outbox_v1` . A phone freezes the tab the moment it is backgrounded, kills the in-flight request, and may evict the tab entirely. An unconfirmed write held only in memory is then lost forever, **and the next boot pulls old server state over it.**

So every save is flagged in the outbox **with its value, before the network attempt**, and cleared only on confirmed success. `localStorage` writes are synchronous, so the flag survives the OS killing the page mid-save. `_obReplay()` replays it through `sSet` , so the merge-safe layer applies and replaying days later is safe.

`_obReplay()` runs before the first load in `_boot` . Push before you pull, or the pull overwrites the thing you never managed to push.

4.4 Serialised writes

`_writeChain[key]` chains saves per key, so two quick saves cannot interleave their read-merge-write and lose one of them.

4.5 Failed writes are visible

`_pendingWrites` plus a fixed red banner: "Changes not saved to the server yet, retrying automatically." A 25-second interval retries. The old behaviour reported success while the change had never left the device, which is how you lose trust in a system permanently.

4.6 Background refresh, and when it must not run

`refreshFromServer()` runs every 60 seconds while visible, and `_pushThenRefresh()` runs on `visibilitychange` , `focus` , `pageshow` (bfcache) and `online` . Because rendering blows away the DOM, refresh **refuses** to run when: the gate is showing, any modal is open, there are unsynced local edits, or the focused element is an input, textarea or select. Removing any of those guards will yank state out from under someone mid-typing.

Note the order in `_pushThenRefresh` : **push first, then pull.** Waiting for the 25-second retry timer is too late on mobile.

SECTION

5. The six decisions that look odd and are load-bearing

5.1 Boot is two-phase on purpose. Do not re-serialise it.

The sign-in card paints first and data loads behind it.

5. The six decisions that look odd and are load-bearing

- `S.screen='gate'; render();` is **the last statement in the script**, not near the top. `renderGate` reads `APP_VERSION`, which is declared late, so painting earlier throws. If you move the first paint up, move the constant too.
- `_cfg` awaits the Supabase library then loads the three config keys **together**.
- `_boot` awaits `_cfg`, replays the outbox, then runs `loadAll()`, `loadAllPortfolio()` and `loadHouse()` **concurrently**.
- Lost leads load lazily, only on the Not Successful and Recovery tabs.
- The won-deal backfill runs after render, not before it.
- The Supabase script is `defer`; fonts are non-blocking; an eight-second cap means a dead CDN degrades to offline mode rather than holding the app hostage.

Measured worst case, CDN unreachable: **13.7 seconds to first paint became 0.47 seconds.**

Two waits are load-bearing. Keep them:

- **`cfgReady()` before any PIN screen.** If `S.pins` arrives late, a PIN that is set looks unset, and the app walks the RM through overwriting it.
- **`bootReady()` before the app opens.** A half-loaded pipeline that the RM then saves writes an empty book over the real one.

`markLost` awaits `loadLost` first for the same reason.

5.2 `seedIfEmpty` has three gates, and it needs all three

`seedIfEmpty` writes the built-in starter book **over the server row**. It exists only to furnish a brand-new database.

It fired against a live database in August 2026 and RMs lost the leads they had loaded. The mechanism is worth stating exactly, because it is the general lesson in this codebase:

> A 30 July boot rewrite replaced the seed's per-RM re-read with a check against already-loaded state, treating the read as pure cost. The read was carrying safety. When a Supabase read fails, `sGet` falls back to `_mem`; **`const _mem = {}` is in-memory and empty on every page load**; so `loadRM` stores `[]`, which is indistinguishable from an empty table; and the seed wrote itself over a live book.

Three gates now, and **all three must hold** before a single row is written:

1. there is a Supabase client at all;
2. `_readOk[key]` is true, that key's read was actually answered by the server rather than fallen back;
3. a **fresh re-read immediately before writing** still shows the row empty.

Gate 3 costs reads only on the seed path, which on a live database never runs.

The general rule, and it generalises past this function: a failed read and an empty table look identical downstream. Anything that *writes* on emptiness needs positive proof of emptiness. Any future "skip the read to make boot faster" change has to answer this first.

(The source comments disagree on the date, one says 6 August and one says 12 August 2026. The remediation is dated 12 August. The mechanism is not in doubt.)

5.3 `_guardWrite`: the structural fix, not another cleanup tool

Everything else in the file tries to be careful. `_guardWrite` assumes something got through anyway. It sits in `sSet` and **refuses** the write, telling the user rather than failing silently.

On the collection keys only, matching `/^(deals|portfolio|lost):/`:

- refuse a write that would **empty** a non-empty collection;
- refuse a write that would leave **half or fewer** of its rows in one go (`<=`, since 22 Sept 2026).

With two deliberate carve-outs:

- it only bites at **three or more existing rows**, so an RM with two leads may delete both, one at a time, without tripping an alarm;
- the halving rule needs **four or more**.

Bulk archive is unaffected, because archiving does not shorten the array.

This would have stopped the August loss outright. A save that empties an RM's book, or throws most of it away at once, is never something a person did on purpose. An RM deletes one lead at a time.

SECTION · CONTINUED

5. The six decisions that look odd and are load-bearing

Its known limit, and it is real: on 2 September 2026 two rows went missing from an RM's book and that is **below the threshold**, so it passed. The guard is a floor, not a fence.

A second gap was real and is closed. Until 22 September 2026 the rule was `<` half, so a book going from six rows to three (exactly half) passed by one row. That is how an RM's book was overwritten on 21 September 2026 without the guard firing. It is now `<=`. Deleting one row at a time still works at every size, because each single step stays above the line.

5.4 `_snapshot`: exact rows, not a reconstruction

Before every overwrite of a guarded key, the previous value is copied to `config:snapshot:<key>`, at most once per key per ten minutes, fire and forget so it never delays the save.

The reason it exists alongside Recovery: **Recovery reconstructs** a book from journals and payroll months, and a reconstruction loses notes and next steps. **A snapshot is the exact rows back.**

⚠️ **Snapshots only exist from 22 September 2026.** From 12 August to 22 September every snapshot write failed silently: `_snapshot` passed `_route(…)` into `_sbSetAdopt`, which routes its key itself, so the second `_route` threw on an object and an empty `catch` swallowed it. The database held zero snapshot rows. The call now passes the key, and a failure is logged with `console.warn`. Anything lost before 22 September can only be rebuilt through Recovery (section 6).

5.5 `_livePremium`: a won deal exists twice and the two records drift

`_promoteWonToBook` copies a stage-7 deal into the portfolio and then archives the deal. So the same client exists as two records: a **pipeline record frozen at the signing figure**, and a **book row that is what anybody actually maintains afterwards**. Payroll reads the book, Command View edits the book inline.

When a premium was corrected in the book from R460,634 to R4,606.34, "Biggest deal of the month" carried on quoting the stale deal record.

`_livePremium(slug, deal)` resolves a won deal's premium through the portfolio by client name, using the same case and whitespace insensitive matcher payroll uses (`_bookKey`), and falls back to the deal's own premium when the client is not on the book yet.

It is applied to **all three My Day sites**: `_titles`, `_bestMonth` and both month-total reducers. That is the point. A corrected figure must not show in one place and not another. Archived book rows still resolve, because the same sweep that archives the deal would otherwise reinstate the stale number.

5.6 The payroll rules that look arbitrary

The formula changed on 27 August 2026: `payout = commission / 2`. The commission statement now gives a rand commission per client, so there is no rate percentage and no broker fee in the payout. `RM_COMMISSION_SHARE = 0.5`.

Legacy rows keep the old formula, and this is not tidy-up debt. Anything captured before 27 August has no `commission`, only `premium` and `commRate` and possibly `brokerFee`. `payoutOf` falls back to `premium × commRate% + brokerFee / 2` whenever `commission` is absent, because **a month that has already been paid may never move.** `BROKER_FEE_SHARE` and `feeShareOf` exist for exactly that. Do not delete them. The payroll screen tags those rows "legacy".

`_isOpenMonth(mo)` freezes every month except the current one and the one before it. Not "current month only", because payroll is closed *after* month end: August is run during September, so a strict rule would freeze the very month being worked on. Premium and commission sync from the book only inside that window. **Policy number and UMA are descriptive rather than financial and may be corrected on any month.** This protects the payroll snapshots that Recovery rebuilds a lost book from.

`syncPayrollFromBook(pay, force)` fills blanks only. A month seeds from the book **once**, when it is first opened. That was the "broker fee does not pull through" bug: a month opened before a fee was captured kept its seeded zero forever. So on every load, blanks are filled; and a "Refresh from Active Clients" button force-syncs fee, policy and UMA **and adds clients won since the month was opened**.

Three details in there that are deliberate:

- **Premium and rate are never auto-synced**, only force-synced, because they are edited per month and re-reading them would wipe the work.
- A fee typed on the payroll screen sets `feeEdited` so it is never silently overwritten. **A deliberate R0 stays R0.**
- Matching is RM plus client name, case and whitespace insensitive, because payroll rows carry no portfolio id. Archived clients are excluded.

SECTION

6. Recovery reads two records, and neither is enough alone

`recoveryAnalyse()` proposes restorations as tick boxes. It draws on two sources because they cover different things:

6. Recovery reads two records, and neither is enough alone

1. **The change journal** (`changes:<date>:<slug>`) rebuilds **pipeline deals**. It replays every entry into a desired state per client: added, stage, health, archived, lost, restored, won, deleted. A client marked `delclient` is never resurfaced.
2. **Payroll months** (`payroll:<YYYY-MM>`) rebuild **Active Clients**. Every month is seeded from the RMs' books, so each month is a **dated snapshot of the book**, carrying client, premium, rate, broker fee, policy and UMA. That is the only surviving record when `portfolio` is overwritten. It takes the most recent month mentioning each client and skips ones already back on the book.

Then a third fallback, which exists because of a real miss: a payroll month only holds the clients that existed when it was **first opened**, so anyone won since then appears in no snapshot at all. Won deals at stage 7 are therefore a second source for `mkclient` .

Before this was built, Recovery only *reported* missing Active Clients and told the user to "re-import or add manually", which is why one RM's won clients stayed missing after the first restore.

SECTION

7. Do not do these things

- Do not add a delete-all-then-reinsert save path.
- Do not remove a read on the grounds that the data is already in memory, unless you have checked whether that read is the thing distinguishing "failed" from "empty".
- Do not weaken `_guardWrite` to make a bulk operation convenient. Make the bulk operation not shorten the array.
- Do not remove `cfgReady()` or `bootReady()` .
- Do not delete `BROKER_FEE_SHARE` , `feeShareOf` or the legacy branch of `payoutOf` .
- Do not let `refreshFromServer` run with a modal open or an input focused.
- Do not ship without bumping `APP_VERSION` .
- Do not give RMs a restore button.
- Do not wrap a safety write (snapshot, journal, outbox) in an empty `catch` . Log the failure.

SECTION

8. Known state and open items

- **Two rows went missing from an RM's book on 2 September 2026** and were restored from Command View. Root cause not traced. If it recurs, get the date, time and device from the RM and read `changes:<date>:<slug>` .
- **An RM's book went from six clients to three on 21 September 2026.** That traced to the halving gap in 5.3, now closed. No snapshot existed to restore from (5.4); rows came back through Recovery.
- `users` and `activity` tables exist and are unused.
- RM slugs are hard-coded in a `RMS` array in the source, including one whose display name is spelled differently from the settled spelling of the person's name. Changing a slug orphans that RM's rows, since every key is `deals:<slug>` . Rename the display name, not the slug.
- Manager passcodes and operator master keys are hard-coded constants in the page source. See document 2.
- Suggested next work, roughly 30 hours, is in `deal-notes/rm-system-roadmap.md` : nightly data-health alert, one-tap undo per RM from snapshots, commission statement import, renewal calendar, version nag. **Supabase Auth should precede all of it.**

SECTION

9. Changes since the first issue (21 Sept 2026)

- **21 Sept 2026, `942f4e0` :** operator master keys labelled by role in the source, not by a person's name.
- **22 Sept 2026, `31fad50` :** `_snapshot` fixed so snapshots actually write, failures logged; `_guardWrite` halving rule `<` → `<=` ; `APP_VERSION` `v2026.09.22a` .
- **30 Sept 2026, `ba7b48d` :** a separate `conveyancing/` folder removed from the repo (55 files). No app code changed.